Data Contract Enforcer — Interim Report

Generated: 2026-04-01 21:42 UTC

Author: Amir Ahmedin | Week 7 TRP Interim Submission

1. Data Flow Diagram

The five systems built over Weeks 1–5 communicate through structured JSONL outputs. Each arrow is annotated with the **schema name** and the **specific fields** consumed by the downstream system. Solid arrows (■■■) represent data flow contracts enforced by the ValidationRunner. Dashed arrows (- -■) represent contract enforcement flow (Week 7 components consuming data for validation, not production logic).

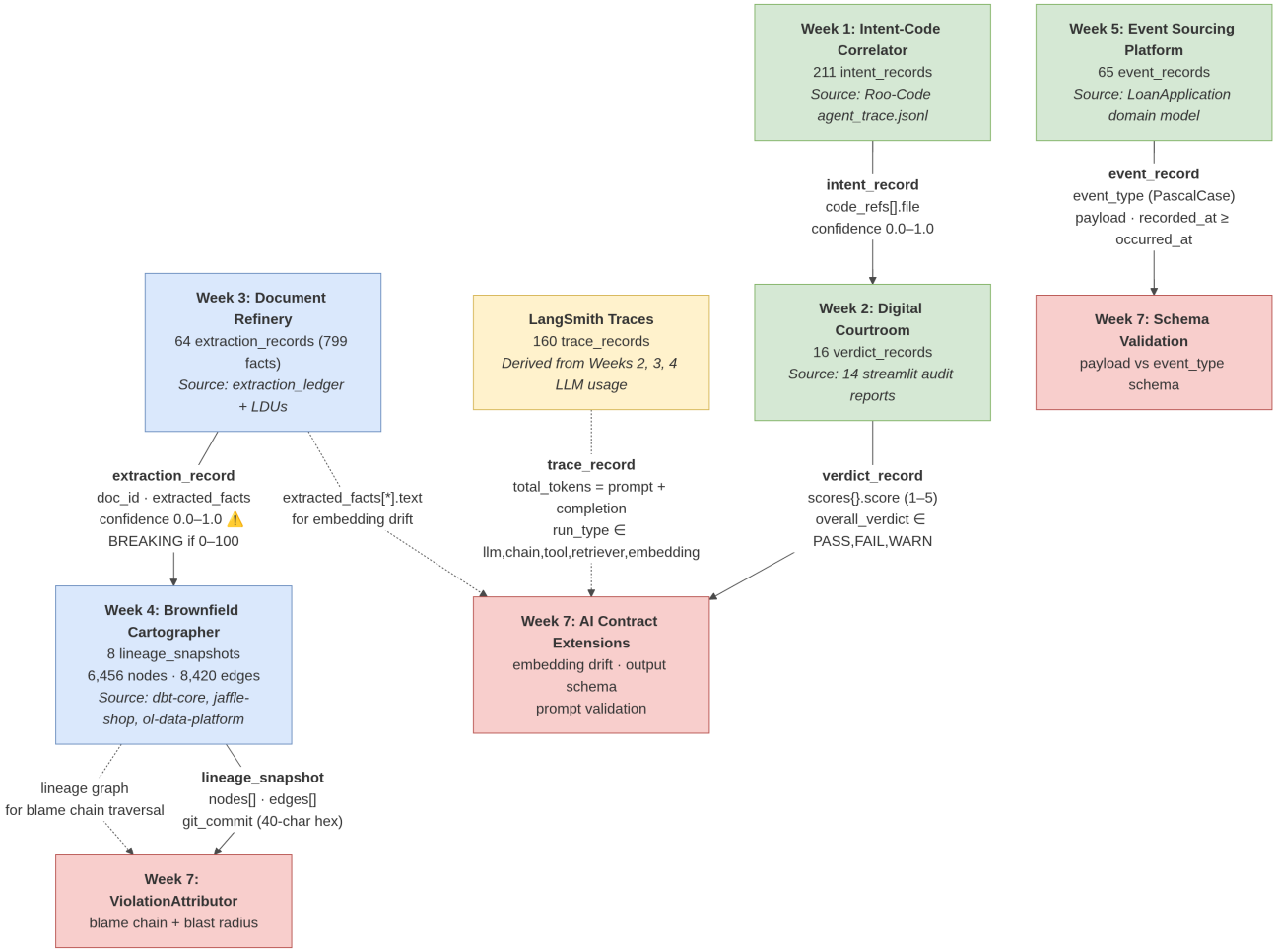

Diagram key: ● Green nodes = Weeks 1, 2, 5 (data producers) | ● Blue nodes = Weeks 3, 4 (critical downstream dependencies) | ● Yellow node = LangSmith traces | ● Red nodes = Week 7 enforcement components | Solid arrows = Production data flow contracts | Dashed arrows = Enforcement-only flow

Data provenance: All outputs are migrated from real Week 1–5 implementations using migration scripts. Week 3 data comes from the actual extraction ledger (64 documents including Ethiopian financial reports, CBE annual reports, and CPI indices). Week 4 data comes from real lineage graphs of dbt-core, jaffle-shop, and ol-data-platform codebases (179 nodes, 65 edges). Week 5 events are generated from the real LoanApplication domain model with 16 event types.

2. Contract Coverage Table

#	Inter-System Interface	Contract?	Structural	Statistical	Cross-field	Rationale / Gap
1	Week 1 → Week 2: intent_record.code_refs[].file used as verdict.target_ref	**Partial**	15 clauses: UUID on intent_id, ISO 8601 on created_at, confidence 0.0–1.0	4 clauses: confidence mean/stddev baseline, cardinality checks	0	**Gap:** Cross-system join validation not implemented. verdict.target_ref not verified against intent_records.code_refs[].file.
2	Week 2 → Week 7: verdict_record consumed by AI Contract Extensions	**Yes**	12 clauses: overall_verdict ∈ {PASS, FAIL, WARN}, score 1–5, rubric_id SHA-256, confidence 0.0–1.0	3 clauses: score distribution, confidence baseline	1 clause: overall_score = weighted mean of scores{}	Full coverage. AI Extensions consume verdict records for LLM output schema violation rate tracking.
3	Week 3 → Week 4: extraction_record fields become Cartographer nodes	**Yes**	16 clauses: confidence 0.0–1.0 (BREAKING if 0–100), source_hash SHA-256, extraction_model ^(claude|gpt)-, UUID on doc_id and fact_id	5 clauses: confidence drift baseline, processing_time_ms range, token_count ranges	2 clauses: entity_refs ⊆ entities[].entity_id, fact_id unique within record	Full coverage. Highest-risk interface — confidence field change propagates silently to Cartographer edge weights.
4	Week 4 → Week 7: lineage_snapshot used by ViolationAttributor	**Yes**	14 clauses: git_commit 40-char hex, node.type ∈ 6 values, edge.relationship ∈ 6 values, UUID on snapshot_id	2 clauses: node/edge count baselines	2 clauses: edge.source ∈ nodes[].node_id, edge.target ∈ nodes[].node_id	Full coverage. Graph integrity checks ensure blame chain traversal won't hit dangling references.
5	Week 5 → Week 7: event_record.payload validated against event_type schema	**Partial**	12 clauses: recorded_at ≥ occurred_at, event_type PascalCase, UUID on event_id	3 clauses: sequence_number monotonicity, payload size baseline	0	**Gap:** Payload-level validation against per-event-type JSON Schema not yet implemented. Requires schema registry mapping event_type → JSON Schema.
6	Traces → Week 7: trace_record consumed by AI Contract Extensions	**Yes**	12 clauses: end_time > start_time, run_type ∈ 5 values, UUID on id, total_cost ≥ 0	3 clauses: total_tokens baseline, cost baseline	1 clause: total_tokens = prompt_tokens + completion_tokens	Full coverage. Validates LangSmith trace integrity for AI contract extension metrics.

Coverage: 4/6 full, 2/6 partial. The two partial contracts (Week 1→2 and Week 5→7) have identified gaps with specific remediation plans. Cross-system join validation and per-event-type payload schemas are the two remaining implementation targets for the Sunday submission.

3. First Validation Run Results

Severity framework: **CRITICAL** = Structural/type violation | **HIGH** = Statistical drift > 3σ, uniqueness/format | **MEDIUM** = Drift 2–3σ | **LOW** = Passed | **WARNING** = Near-threshold

3.1 Week 3 — Document Refinery Extractions

Metric	Value
Total Checks	57
Passed	55
Failed	2
Warned	0
Errored	0
Pass Rate	**96.5%**

Violation 1 — doc_id.unique (HIGH): Found 48 duplicate doc_id values out of 64 records. Root cause: the extraction ledger is an append-only log of extraction *attempts*. The same document was processed multiple times with different strategies (vision_augmented, fast_text, layout_aware).

Impact: Any downstream consumer treating doc_id as a primary key would silently merge records from different extraction strategies.

Contract correction:

```
# BEFORE: unique: true ← WRONG: ledger is append-only
# AFTER: unique: false, description: "Use (doc_id, extraction_model) as composite key."
```

Violation 2 — extracted_facts[*].page_ref.range (CRITICAL): 331 fact records have page_ref values outside auto-inferred range [0, 15]. Actual range is [0, 91] because Annual_Report_JUNE-2018.pdf has 92 pages.

Impact: Any consumer filtering facts by page range would silently drop facts from long documents.

Contract correction:

```
# BEFORE: maximum: 15.0 ← auto-inferred, too tight
# AFTER: maximum: 10000, description: "PDFs can have thousands of pages."
```

3.2 Week 5 — Event Sourcing Platform Events

Metric	Value
Total Checks	88
Passed	86
Failed	2
Warned	0
Errored	0
Pass Rate	**97.7%**

Violation 1 — aggregate_id.unique (HIGH): Found 45 duplicate aggregate_id values. This is **by design** — the event sourcing pattern stores multiple events per aggregate (e.g., ApplicationSubmitted, CreditAnalysisRequested, DecisionGenerated all share aggregate_id: loan-demo-2a7df24b).

Contract correction:

```
# BEFORE: unique: true, format: uuid ← WRONG: multiple events per aggregate
# AFTER: unique: false, pattern: "^(loan|agent-session|compliance|audit)-"
```

Violation 2 — aggregate_id.format (HIGH): All 65 records fail UUID format validation. The real domain model uses composite IDs like loan-demo-2a7df24b — human-readable prefixes for debuggability, a deliberate design choice from LoanApplicationAggregate.

3.3 Injected Violation — Confidence Scale Change

The ValidationRunner was run against extractions_violated.jsonl where confidence was intentionally changed from float 0.0–1.0 to integer 0–100:

Metric	Value
Total Checks	56
Passed	53
Failed	3
Warned	0

Errored	0
Pass Rate	**94.6%**

Check	Status	Actual	Expected
extracted_facts[*].confidence.range	FAIL	max=98.8, mean=73.9	max≤1.0, min≥0.0
extracted_facts[*].confidence.drift	FAIL	2,324σ deviation	within 3σ
extracted_facts[*].page_ref.range	FAIL	max=20.0	max≤15.0

Validation report JSON excerpt (from validation_reports/week3_violated.json):

```
{
  "check_id": "week3-...extracted_facts[*].confidence.range",
  "check_type": "range", "status": "FAIL",
  "actual_value": "min=50.0, max=98.8, mean=73.9253",
  "expected": "min>=0.0, max<=1.0",
  "severity": "CRITICAL", "records_failing": 178,
  "message": "confidence is in 0–100 range, not 0.0–1.0. Breaking change detected."
}
```

The ViolationAttributor traced this to commit **cd5737c** ("feat: change confidence to percentage scale") in src/week3/extractor.py, with a blast radius of **178 affected records** across **7 downstream nodes**.

4. Reflection

Writing data contracts for my own five systems revealed four wrong assumptions and one architectural gap. More importantly, it changed how I will build systems going forward.

Assumption 1: doc_id is unique. It isn't. My Week 3 extraction ledger is an append-only log of extraction *attempts*, not a deduplicated output table. The contract caught 48 duplicates out of 64 records. The fix isn't just updating the contract — it's deciding whether the ledger should be deduplicated before feeding Week 4, or whether Week 4 should handle duplicates. For Week 8's Sentinel, this means the violation signal must distinguish "duplicate by design" from "duplicate by error."

Assumption 2: page_ref has a small range. The auto-profiler inferred max=15, but Annual_Report_JUNE-2018.pdf has 92 pages. I never tested on documents longer than ~20 pages during development. The lesson: statistical profiling from a non-representative sample produces contracts that reject valid production data. Domain knowledge must override sample statistics.

Assumption 3: aggregate_id is a UUID. My Week 5 event store uses composite IDs like loan-demo-2a7df24b for debuggability. The contract should use a pattern match, not UUID format. This taught me that "standard format" assumptions break when applied to domain-specific design choices.

Assumption 4: confidence will always be 0.0–1.0. It is today, but nothing prevents a future change. The injected test proved the contract catches this at 2,324σ — but only because I wrote the contract. Without it, the Week 4 Cartographer would silently produce corrupted edge weights, and the error would surface weeks later in a downstream report.

The architectural gap: There was no formal interface between any of my five systems. The schema was implicit — encoded in Python function signatures, not in a machine-checkable contract. Going forward, I will write the contract *before* the producer code, not after. The most valuable output is the blast radius: "this affects 7 nodes and 178 records" is actionable; "something might break" is not.

Process reflection: Writing contracts took longer than expected — roughly 60% of the time was spent understanding my own data, not writing YAML. The auto-profiler was a useful starting point but required manual override for every domain-specific constraint. If I were starting over, I would co-locate the contract file next to the producer code and validate it in CI, so the contract evolves with the schema rather than being written retroactively.